Design and Implementation of a 5-bit Carry Look Ahead Adder

ARUN VENKAT
International Institute of Information Technology, Hyderabad
ROLL NO: 2024102067
Email: arunvenkat.jonna@students.iiit.ac.in

***Summary*—**This project is about designing a fast 5-bit Carry Look-Ahead (CLA) adder for efficient binary addition. The circuit is built using different blocks: one to create signals, one to handle the carry logic, and one to calculate the final sum. The design works with a clock signal to keep everything in sync—inputs must be ready before the clock goes high, and the results appear by the next time the clock goes high. To store data and manage the timing, we used D flip-flops designed with a specific transistor size ratio of $W_p/W_n = 20\lambda/10\lambda$.

I. Introduction

Arithmetic operations, particularly addition, are fundamental to the operation of digital processors and signal processing systems. As the demand for computation speed increases, the efficiency of these arithmetic circuits becomes critical.

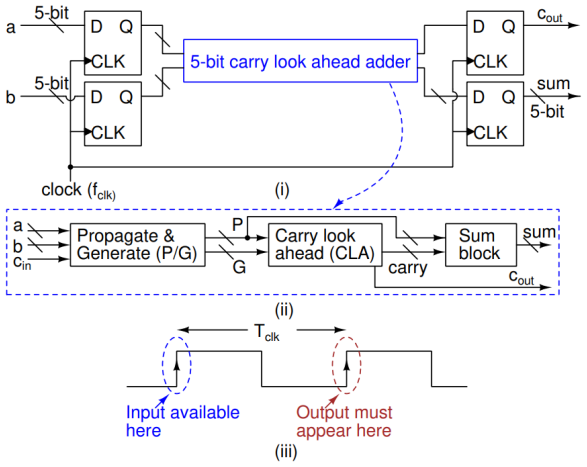

Fig. 1. Circuit Diagram

In a conventional Ripple Carry Adder (RCA), the carry-out of each bit position depends on the carry-in from the previous bit. This creates a linear delay chain that grows with the number of bits, limiting the overall performance.

To overcome this latency, this project implements a **5-bit Carry Look-Ahead (CLA) Adder**. The CLA architecture employs a parallel computation scheme using Propagate (P_i) and Generate (G_i) functions to determine carry signals simultaneously for all bit positions, rather than waiting for the ripple effect.

A. Project Scope

This project covers the full VLSI design flow, including:

- **Logic Design:** Implementation of modular blocks using Static CMOS logic for robust noise immunity and low static power.
- **Synchronization:** Utilization of Transmisssion gate D Flip-Flops to pipeline inputs and outputs, optimizing the design for high-speed synchronous operation.
- **Physical Design:** Custom layout generation (Stick diagrams and Magic layout) using 180nm technology rules ($\lambda = 0.09\mu m$).
- **Verification:** Comparison of pre-layout and post-layout simulation results to analyze the impact of parasitic delays.

II. Carry Look-Ahead Adder

We used Carry Look-Ahead (CLA) adder which is a high-speed arithmetic circuit designed to reduce the propagation delay associated with linear ripple-carry adders. Unlike ripple adders, where each stage must wait for the preceding carry bit, the CLA utilizes parallel logic to determine carry signals instantaneously. This is accomplished through the use of Propagate (p_i) and Generate (g_i) functions:

$$p_i = A_i \oplus B_i \quad \text{(Propagate function)} \tag{1}$$

$$g_i = A_i \cdot B_i \quad \text{(Generate function)} \tag{2}$$

The equation for the next carry bit is given by:

$$C_{i+1} = G_i + (P_i \cdot C_i) \tag{3}$$

Here, A_i and B_i are the inputs at the i-th position, and C_i is the carry-in. By computing carries in parallel, the CLA significantly outperforms ripple-carry adders in total computation time, making it the preferred architecture for high-speed processors and digital systems.

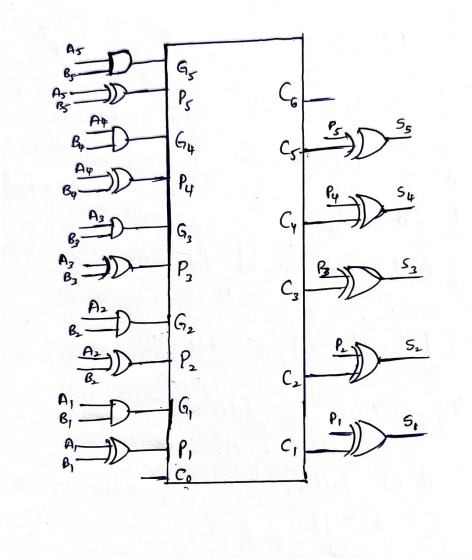

Fig. 2. Adder outline

III. Adder Details

The Carry Look-Ahead (CLA) adder in this project is completely based on CMOS static logic. We used XOR,AND and INVERTOR as basic gates in the design. Each circuit with its sizing is explained in the bottom sections. The circuit using CMOS is given at the end of the report. Below are the equations of C1,C2,C3,C4,C5,C6 in terms of only pi,gi without any carry.

Assuming the initial carry-in $C_1 = 0$, the carry equations are expanded below.

A. Carry Equations (Indices 1-5)

For a 5-bit adder, the carry signals range from C_1 (Input) to C_6 (Output). With $C_1 = 0$, the equations simplify to depend solely on Propagate and Generate terms:

$$C_1 = 0 \quad \text{(Initial Carry-In)} \tag{4}$$

$$C_2 = G_1 \tag{5}$$

$$C_3 = G_2 + (P_2 \cdot G_1) \tag{6}$$

$$C_4 = G_3 + (P_3 \cdot G_2) + (P_3 \cdot P_2 \cdot G_1) \tag{7}$$

$$C_5 = G_4 + (P_4 \cdot G_3) + (P_4 \cdot P_3 \cdot G_2) + (P_4 \cdot P_3 \cdot P_2 \cdot G_1) \tag{8}$$

The final carry-out for the 5-bit system is C_6:

$$C_6 = G_5 + (P_5 \cdot G_4) + (P_5 \cdot P_4 \cdot G_3) + (P_5 \cdot P_4 \cdot P_3 \cdot G_2) + (P_5 \cdot P_4 \cdot P_3 \cdot P_2 \cdot G_1) \tag{9}$$

This logic ensures that C_6 is available immediately based on the input states, without waiting for a ripple effect through the previous 5 stages.

IV. Topology

In the netlist we used:

- $W_n = 10 \times \lambda$
- $W_p = 2 \times W_n = 20 \times \lambda$
- $\lambda = 0.09\,\mu m$

The length of each MOSFET used is $L = 2 \times \lambda$.
All the sizing in such a way that we equate the net charging or discharging resistance to be equal to a standard reference cmos invertor.

A. Inverter

The inverter is implemented using CMOS static logic. Inverters are used to get complimentary inputs and clock bar in circuits, and buffer as providing isolation and strengthening weak signals to drive larger loads.

B. XOR Gate

The XOR gate is also implemented using CMOS static logic. Similar to the other gates, this topology is chosen to ensure low power consumption and robust operation. For the XOR gate sizing, specifically the section where inputs and inverted inputs are processed, the PMOS width is set to $4 \times W$, and the NMOS width is $2 \times W$, relative to the standard inverter width W.

C. AND Gate

The AND gate is designed using a combination of a CMOS NAND gate and a minimum-sized inverter. Like the inverter, it is implemented using CMOS static logic, which is preferred for its low power consumption, high noise immunity, and reliable operation across a wide range of conditions.

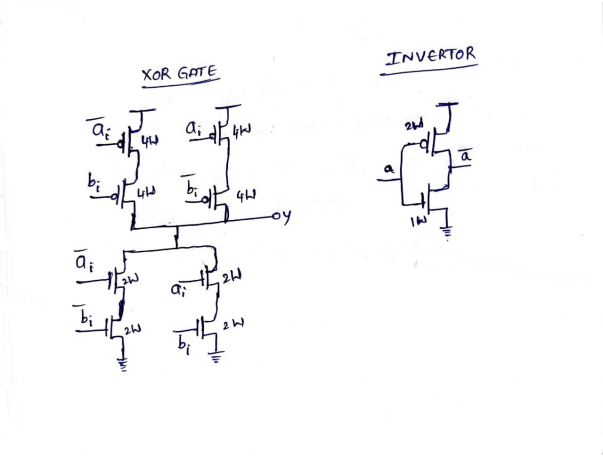

Fig. 3. Invertor and XOR

Regarding sizing, the PMOS width for the AND gate is $2 \times W$, and the NMOS width is $2 \times W$, where W is the width of the NMOS in the standard inverter.

D. D-Flip-Flop (Transmission Gate Logic)

The D Flip-Flop is implemented using a Master-Slave configuration based on CMOS Transmission Gates (TG). This topology was chosen for its robust static operation and reliable data retention.

The circuit consists of two stages: a Master latch and a Slave latch, controlled by complementary clock phases (Clk and $\overline{Clk}$).

- **Master Stage:** Samples the input D when the clock is active and holds the data using a feedback loop.
- **Slave Stage:** Passes the data from the Master to the output Q on the complementary clock phase, ensuring edge-triggered behavior.

Transistor Sizing Strategy: As shown in the circuit schematic, a specific sizing ratio is employed to ensure writability and speed:

- **Forward Path:** The primary inverters are sized with $W_p = 4W$ and $W_n = 2W$ to provide strong drive capability.
- **Feedback Path:** The feedback inverters are designed as "weak keepers" with significantly smaller dimensions ($W_p = 1W$ and $W_n = 0.5W$).

This sizing asymmetry ensures that the input signal driven through the transmission gate can easily overpower the weak feedback inverter, minimizing contention during state transitions and reducing propagation delay. The transmission gates utilize a W_p/W_n ratio of $2W/1W$ to balance the rise and fall times.

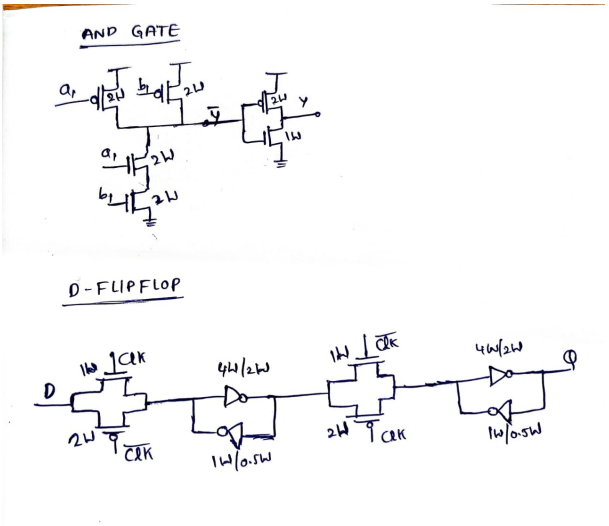

Fig. 4. AND and D-FLIP FLOP

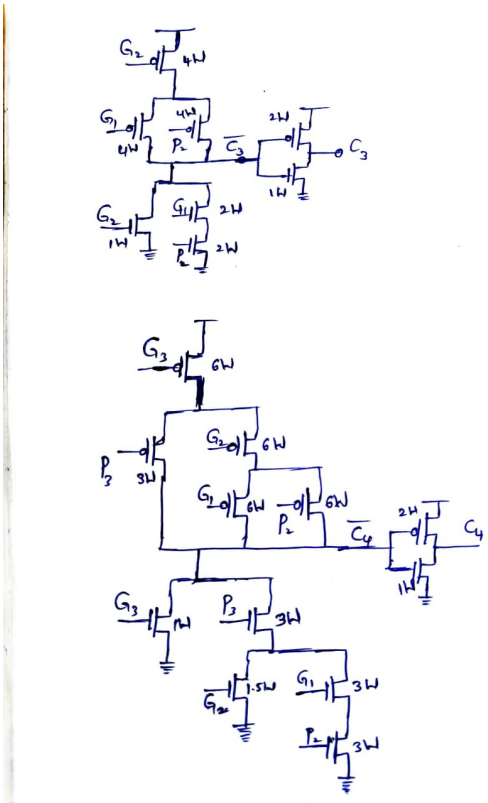

Fig. 5. C3 and C4 CMOS implimentation

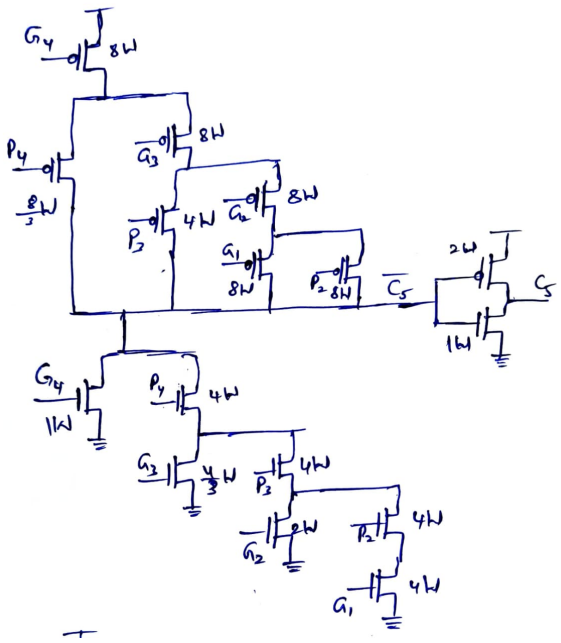

Fig. 6. C5 CMOS implimentation

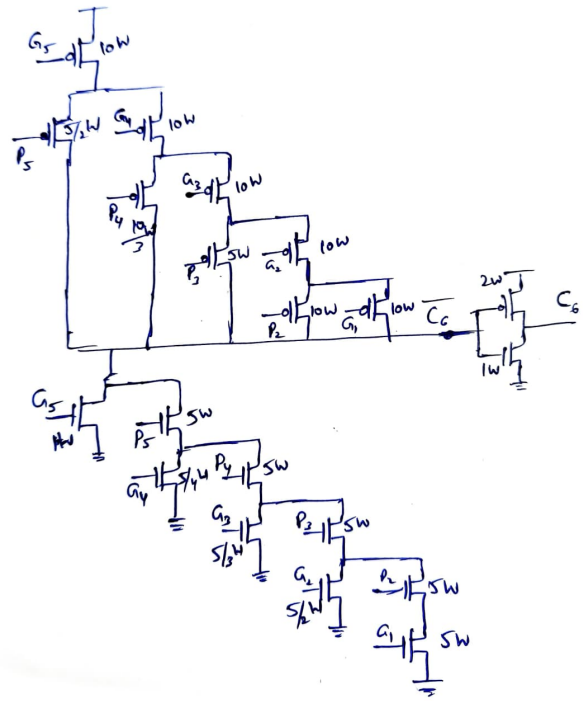

Fig. 7. C6 CMOS implimentation

V. Functionality and times of flip flop

Below are the simulations.

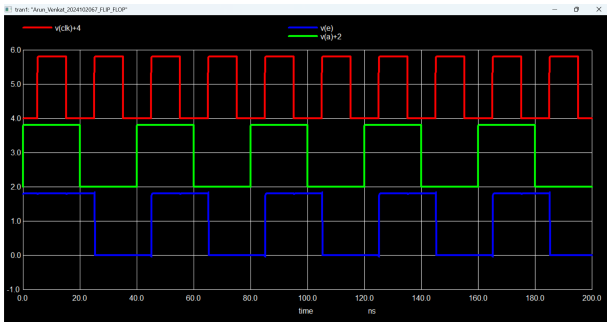

Fig. 8. D-FLIP FLOP TRANSIENT

```
t_a_rise          =   5.01500e-08
t_c_rise          =   5.02563e-08
t_ac_rise         =  1.06309e-10
t_a_fall          =   6.00500e-08
t_c_fall          =   6.01560e-08
t_ac_fall         =  1.06007e-10
t_ac_worst        =  1.06309e-10
t_clk_rise        =   5.00000e-11
t_clkb_fall       =   9.18628e-11
th_pos            =  4.18628e-11
t_clk_fall        =   4.01500e-08
t_clkb_rise       =   4.01939e-08
th_neg            =  4.38908e-11
```

Fig. 9. Setup and hold time

```
t_clk_rise        =   9.00500e-08
t_q_rise          =   9.02078e-08
t_cq_rise         =  1.57764e-10
t_clk_fall        =   6.00500e-08
t_q_fall          =   6.03831e-08
t_cq_fall         =  3.33131e-10
t_cq_worst        =  3.33131e-10
```

Fig. 10. Clk to Q delay

- **Tclk to Q:** Time delay between the clock edge (trigger event) and the output Q stabilizing to its final value.
- t_{pcq} (0 to 1) $\approx 1.57 \times 10^{-10}$ s
- t_{pcq} (1 to 0) $\approx 3.33 \times 10^{-10}$ s
- t_{setup} (0 to 1) $\approx 1.06 \times 10^{-10}$ s
- t_{hold} (1 to 0) $\approx 4.18 \times 10^{-11}$ s

VI. Analysis

A. Timing Analysis

The timing characteristics of the Transmission Gate Flip-Flop, specifically Setup Time (t_{setup}) and Hold Time (t_{hold}), are analyzed by considering the propagation delays and non-ideal clock behaviors (clock overlaps).

1) Setup Time Analysis: The setup time is defined as the time the data must be stable before the active clock edge. In this topology, the setup time corresponds to the delay required for the data to propagate through the first part of the circuit (the Master stage).

$$t_{setup} = \text{Delay of First Part (Master Stage)} \tag{10}$$

The input data D must travel through the first transmission gate and stabilize at the input of the master feedback loop before the transmission gate turns off.

2) Hold Time Analysis: Hold time analysis is critical when considering clock skew or non-ideal clock generation, which leads to overlap periods. We define two overlap regions:

- t_{1-1}: The duration where both $Clk = 1$ and $\overline{Clk} = 1$.
- t_{0-0}: The duration where both $Clk = 0$ and $\overline{Clk} = 0$.

Case 1: During t_{1-1} ($Clk = 1, \overline{Clk} = 1$) In this overlap scenario:

- The first transmission gate (Input TG) remains partially active (NMOS is ON).
- If the input data D_1 transitions (e.g., $1 \rightarrow 0$ or $0 \rightarrow 1$) during this window, the internal node Q_1 changes accordingly.
- This change propagates to D_2. Since the second latch may become active during the transition, this leads to latching wrong data (corruption).
- **Conclusion:** To prevent data corruption, the input must be kept constant during the entire t_{1-1} period.

Case 2: During t_{0-0} ($Clk = 0, \overline{Clk} = 0$) In this overlap scenario:

- The relevant latching mechanism (Latch-2) remains inactive.
- Data changes during this period do not propagate to corrupt the stored state in the same critical manner as Case 1.

Conclusion on Hold Time: Since the corruption of information occurs primarily during the t_{1-1} overlap, the hold time of the circuit is determined by this specific duration.

$$t_{hold} = t_{1-1} \tag{11}$$

It is important to note that the hold time is not the maximum of the two overlaps ($\max(t_{1-1}, t_{0-0})$), but specifically the t_{1-1} duration.

VII. Functionality and Delay of Adder

Below are the simulations.

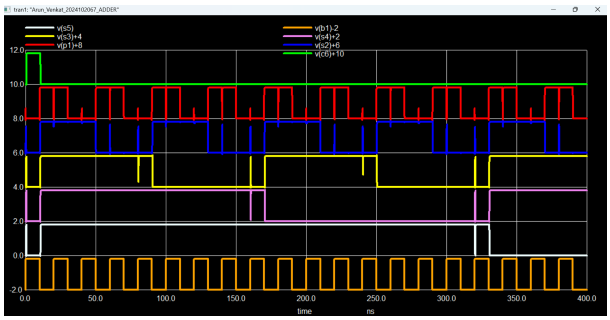

Fig. 11. Adder outputs

```
t_a_rise          =   5.00000e-13
t_c_rise          =   4.99741e-10
t_ac_rise         =  4.99241e-10
t_a_fall          =   1.00015e-08
t_c_fall          =   1.06524e-08
t_ac_fall         =  6.50930e-10
t_ac_worst        =  6.50930e-10
```

Fig. 12. Adder delay

```
t_a_rise          =   1.00015e-08
t_c_rise          =   1.00511e-08
t_ac_rise         =  4.96041e-11
t_a_fall          =   2.00005e-08
t_c_fall          =   2.00258e-08
t_ac_fall         =  2.53326e-11
t_ac_worst        =  4.96041e-11
```

Fig. 13. Adder min delay

The adder ois correctly working from the simulations and the delay of adder comes out be 0.64ns which is considered when the change is at C6 both for rising and falling. The solution to these kinks is that using flip flops.

VIII. Integration Pre-layout

Below are the complete pre layout simulations. Clearly we see for the input A = 11111 and B = 00001 and output is 00000 with C6 = 1.

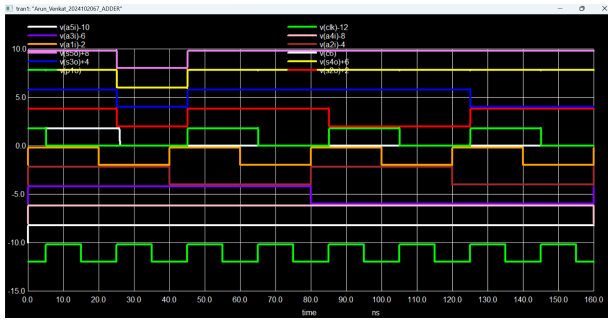

Fig. 14. Adder functionality-1

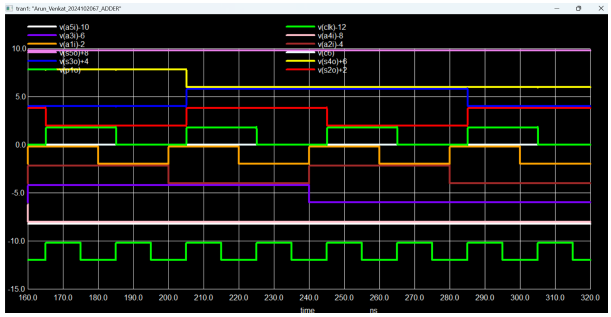

Fig. 15. Adder functionality-2

IX. Worst Case Delay and Maximum Clock Speed

To ensure the correct operation of the synchronous adder design, the clock period T must satisfy the setup time constraint. The data must propagate from the launching flip-flop, through the combinational logic, and arrive at the capturing flip-flop setup window before the next clock edge.

A. Timing Constraint Calculation

The mathematical condition for the minimum clock period is given by:

$$T \geq t_{pcq} + t_{pd} + t_{su} \tag{12}$$

Where:

- t_{pcq} is the Clock-to-Q propagation delay.
- t_{pd} is the worst-case propagation delay of the combinational logic (Adder).
- t_{su} is the Setup time of the D Flip-Flop.

Using the values observed from the analysis:

- $t_{pcq} = 0.2$ ns
- $t_{pd} = 0.64$ ns
- $t_{su} = 0.1$ ns

Substituting these values into the equation:

$$T \geq 0.2 \text{ ns} + 0.64 \text{ ns} + 0.1 \text{ ns} \tag{13}$$

$$T \geq 0.94 \text{ ns} \tag{14}$$

B. Result

The design requires a minimum clock period of **0.94 ns**.

Consequently, the maximum operating frequency (f_{max}) for the design is:

$$f_{max} = \frac{1}{T_{min}} = \frac{1}{0.94 \text{ ns}} \approx 1.06 \text{ GHz} \tag{15}$$

X. Magic Layout

Layouts were designed for the Inverter, AND gate, XOR gate, and the complete CLA circuit.

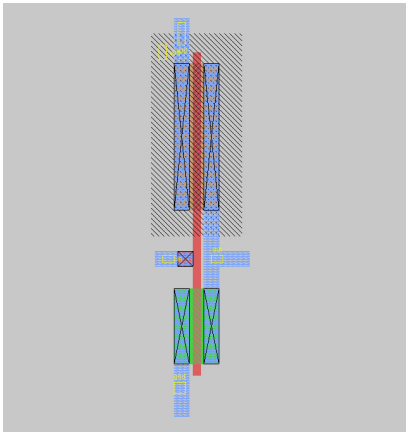

Fig. 16. Layout invertor

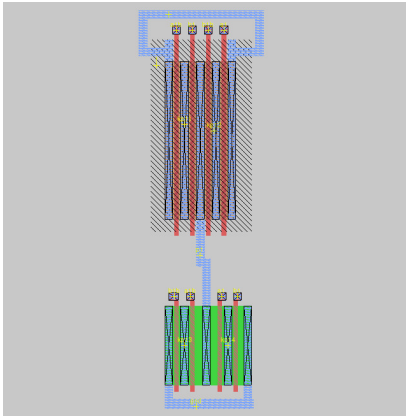

Fig. 17. Layout xor

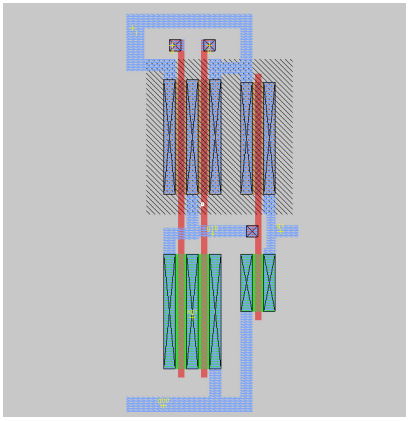

Fig. 18. Layout and

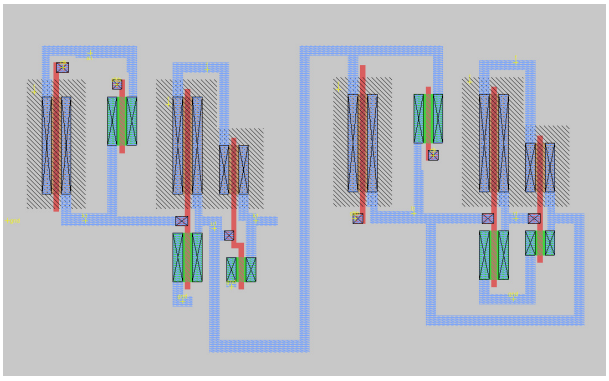

Fig. 19. Layout flip flop

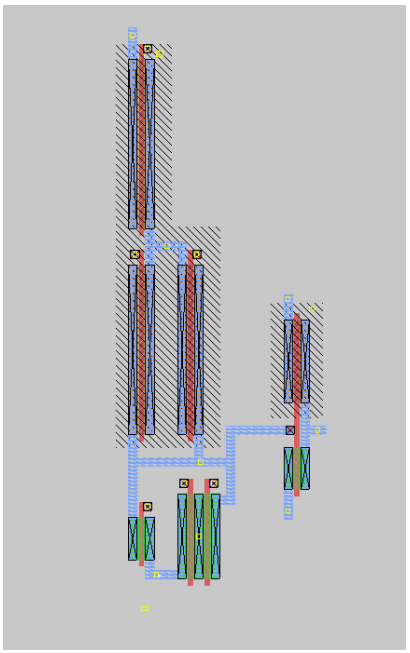

Fig. 20. Layout C3

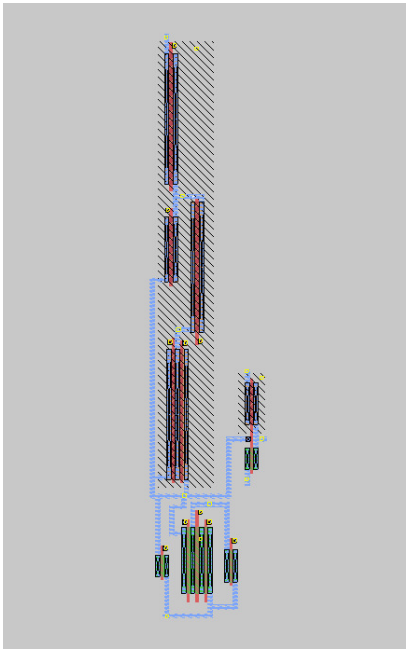

Fig. 21. Layout C4

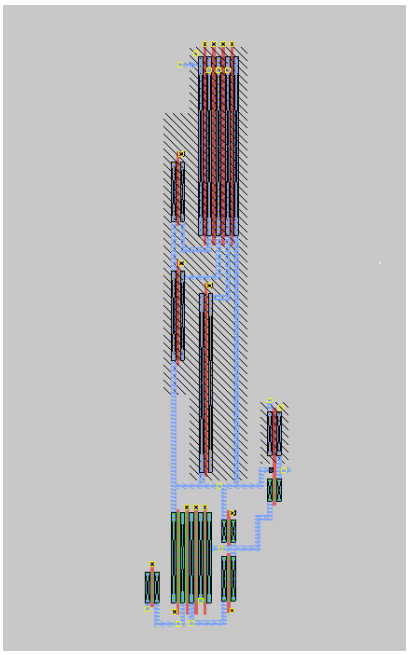

Fig. 22. Layout C5

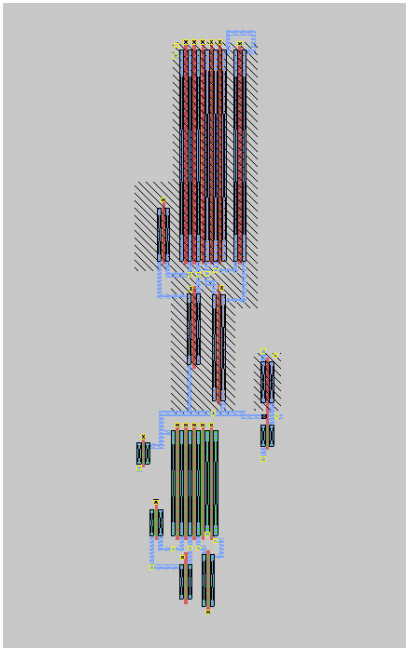

Fig. 23. Layout C6

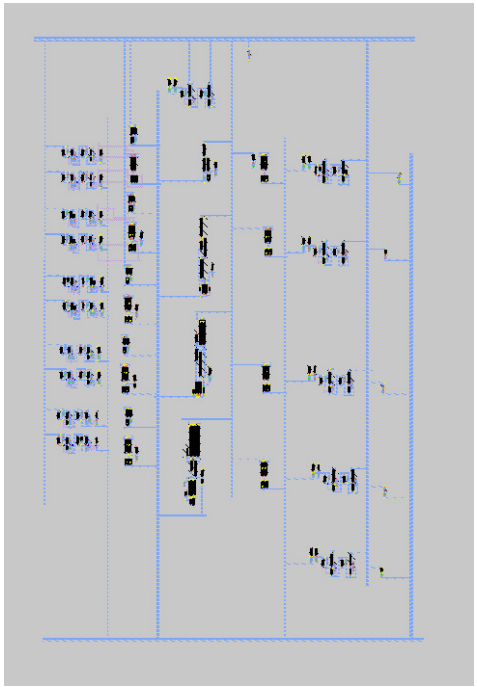

Fig. 24. Final layout

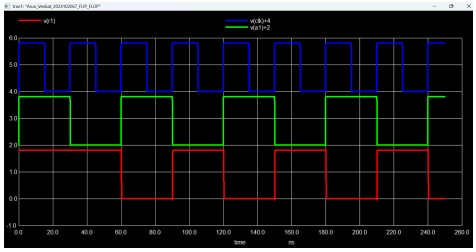

Fig. 25. Post Layout flip flop

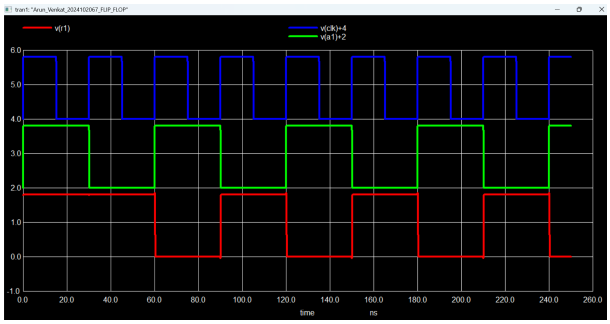

Fig. 26. Post Layout flip flop

XI. Post Layout Simulation

First we will see for flip flop and adder.

Below are the post layout simulation resluts. Clearly we see all the correct outputs. For example for the input A = 11111 and B = 00001 and output is 00000 with C6 = 1. We did the simulation for all changing bits in A and a fixed B which will cover all type of computations.

```
t_a_rise          =   5.01500e-08
t_c_rise          =   5.02370e-08
t_ac_rise         =  8.70032e-11
t_a_fall          =   6.00500e-08
t_c_fall          =   6.01392e-08
t_ac_fall         =  8.92081e-11
t_ac_worst        =  8.92081e-11
t_clk_rise        =   5.00000e-11
t_clkb_fall       =   8.39936e-11
th_pos            =  3.39936e-11
t_clk_fall        =   4.01500e-08
t_clkb_rise       =   4.01878e-08
th_neg            =  3.77938e-11
```

Fig. 27. Flip flop setup and hold time

```
t_clk_rise        =   9.00500e-08
t_q_rise          =   9.01824e-08
t_cq_rise         =  1.32387e-10
t_clk_fall        =   6.00500e-08
t_q_fall          =   6.03393e-08
t_cq_fall         =  2.89262e-10
t_cq_worst        =  2.89262e-10
```

Fig. 28. Layout flip flop

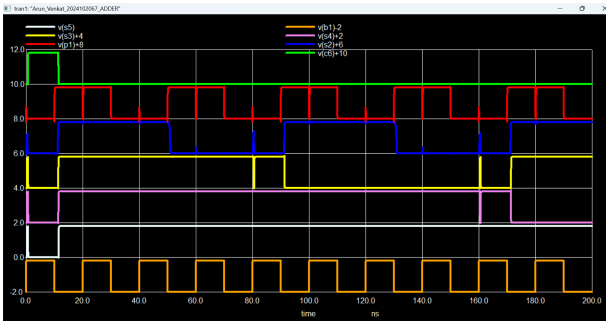

Fig. 29. Adder simulation

```
t_a_rise              =   5.00000e-13
t_c_rise              =   6.41671e-10
t_ac_rise             =  6.41171e-10
t_a_fall              =   1.00015e-08
t_c_fall              =   1.14441e-08
t_ac_fall             =  1.44263e-09
t_ac_worst            =  1.44263e-09
```

Fig. 30. Adder Delay maximum

```
t_a_rise              =   1.00015e-08
t_c_rise              =   1.00511e-08
t_ac_rise             =  4.96041e-11
t_a_fall              =   2.00005e-08
t_c_fall              =   2.00258e-08
t_ac_fall             =  2.53326e-11
t_ac_worst            =  4.96041e-11
```

Fig. 31. Adder Delay minimum

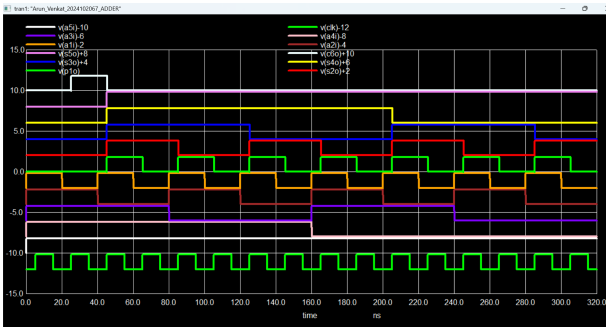

Fig. 32. Post Layout simulation - 1

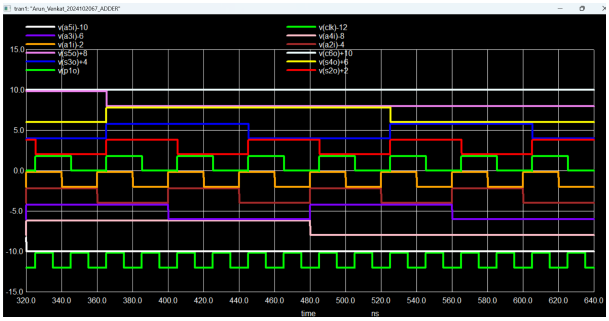

Fig. 33. Post Layout simulation - 2

XII. Simulation Results Comparison

The timing parameters obtained from Pre-layout and Post-layout simulations are summarized in Table I.

TABLE I
Comparison of Pre-Simulation and Post-Simulation Timing Metrics

Parameter	Pre-Simulation	Post-Simulation
Setup Time (t_{su})	100 ps (0.1 ns)	89 ps
Hold Time (t_h)	44 ps	38 ps
Clock-to-Q Delay ($t_{clk\to Q}$)	0.2 ns	0.26 ns
Adder max-Delay (t_{addmax})	0.64 ns	1.44 ns
Adder min-Delay (t_{addmin})	0.044 ns	0.049 ns

A. Post-Layout Worst Case Delay and f_{max}

Following the physical layout and parasitic extraction, the timing constraints were re-evaluated. The post-layout simulation includes additional delays due to interconnect capacitance and resistance.

The minimum clock period condition is:

$$T \geq t_{pcq} + t_{pd} + t_{su} \tag{16}$$

Substituting the post-layout values obtained from simulation:

- $t_{pcq} = 0.26$ ns
- $t_{pd} = 1.44$ ns (Adder propagation delay)
- $t_{su} = 0.089$ ns

$$T_{post} \geq 0.26 \text{ ns} + 1.44 \text{ ns} + 0.089 \text{ ns} \tag{17}$$

$$T_{post} \geq 1.789 \text{ ns} \tag{18}$$

Maximum Frequency Calculation: Based on the critical path delay of 1.789 ns, the maximum operating frequency for the post-layout design is:

$$f_{max(post)} = \frac{1}{T_{post}} = \frac{1}{1.789 \times 10^{-9}} \approx 559 \text{ MHz} \tag{19}$$

This reduction in frequency compared to the pre-layout estimate is expected due to the inclusion of realistic interconnect parasitics.

B. Hold Time Violation Analysis

To ensure reliable operation, the hold time constraint must be satisfied. A hold violation occurs if the new data arrives at the capturing flip-flop too quickly, overwriting the previous data before it is safely latched.

The condition to prevent a hold violation is that the hold time (t_h) must be less than the minimum contamination delay of the path:

$$t_h \leq t_{pcq} + (t_{pd})_{min} \tag{20}$$

Where:

- t_h: Required Hold time of the flip-flop (38 ps from Post-Layout simulation).
- t_{pcq}: Clock-to-Q delay of the launching flip-flop (260 ps).
- $(t_{pd})_{min}$: Minimum propagation delay of the combinational logic (49 ps).

Substituting the values into the inequality:

$$38 \text{ ps} \leq 260 \text{ ps} + 49 \text{ ps} \tag{21}$$

$$38 \text{ ps} \leq 309 \text{ ps} \tag{22}$$

Conclusion: Since the required hold time (38 ps) is strictly less than the minimum data arrival time (309 ps), the condition is satisfied.

Calculating the Hold Slack:

$$\text{Slack} = 309 \text{ ps} - 38 \text{ ps} = +271 \text{ ps} \tag{23}$$

The positive slack confirms that there is **no hold time violation** in the post-layout design.

XIII. Floor Plan of Complete Circuit and Stick Diagrams

The floor plan was designed to organize the layout efficiently, focusing on minimizing silicon area and interconnect delays. The placement follows a structured datapath approach:

- **Input Stage:** The input D flip-flops are placed at the entry point to synchronize the arriving A and B data bits.
- **Logic Core:** The Propagate (P) and Generate (G) logic blocks are situated adjacent to the input stage to immediately process the signals.
- **Carry Chain:** The critical Carry Look-Ahead (CLA) blocks are centrally located to minimize the distance the high-speed carry signals must travel between stages.
- **Output Stage:** The Sum generation blocks and output flip-flops are placed at the end of the datapath to latch the final results.

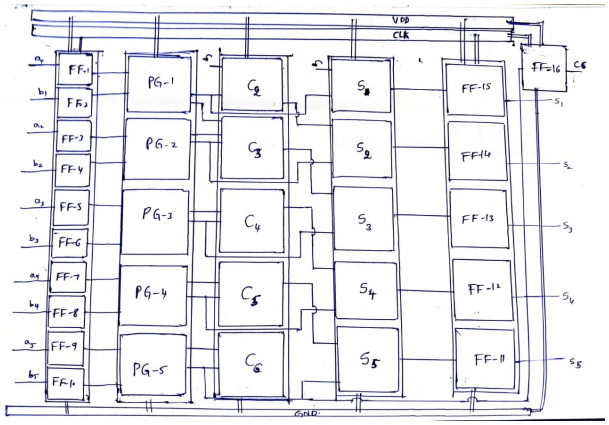

Fig. 34. Layout outline

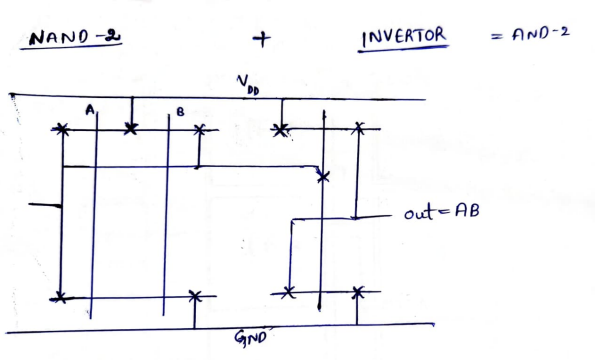

Fig. 35. AND stick diagram

XIV. Verilog HDL Implementation

To verify the logical correctness of the design before physical layout, the 5-bit Carry Look-Ahead Adder was modeled using Verilog HDL. The coding style adopted is **Structural Modeling**, which strictly reflects the hardware architecture designed in the schematic phase.

A. Design Description

The Verilog design is partitioned into three main hierarchical modules:

- **D Flip-Flop Module:** A behavioral model of the edge-triggered register used for input and output synchronization.
- **CLA Logic Module:** This module implements the combinational logic for the Propagate (P), Generate (G), and Carry (C) equations derived in Equation (5-9). It computes the 5-bit Sum ($S[4:0]$) and the final Carry-Out (C_{out}).
- **Top Module:** Instantiates the input registers, the CLA combinational block, and the output registers.

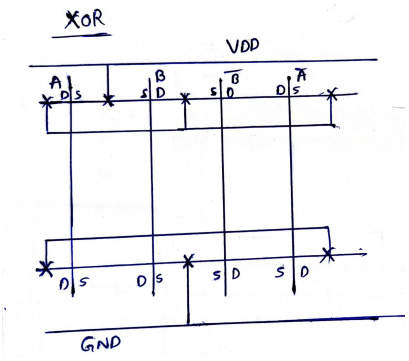

Fig. 36. XOR stcick diagram

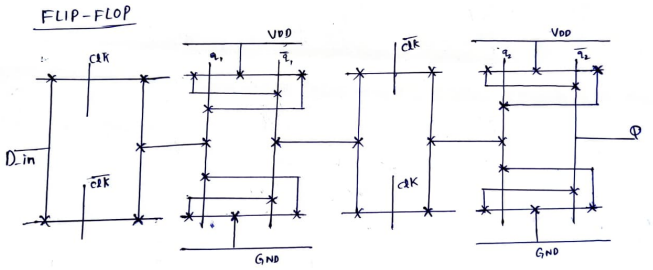

Fig. 37. Flip flop stcick diagram

This structure ensures that the design mimics the pipelined behavior of the physical circuit.

B. Simulation Strategy

A testbench was developed to validate the adder across various input vectors. The simulation checks for:

- **Correctness:** $Sum = A + B + C_{in}$.
- **Latency:** Verifying that if inputs are applied at clock cycle N, the corresponding outputs appear at clock cycle $N + 1$ (due to the output registers).

```
VCD info: dumpfile cla_adder_waves.vcd opened for output.
Time=11000 | A=00000 B=00000 Cin=0 | Sum=xxxxx Cout=x
Time=31000 | A=00011 B=00100 Cin=0 | Sum=00000 Cout=0
Time=51000 | A=01111 B=01111 Cin=0 | Sum=00111 Cout=0
Time=71000 | A=11111 B=00001 Cin=0 | Sum=11110 Cout=0
Time=91000 | A=01010 B=01010 Cin=1 | Sum=00000 Cout=1
Time=111000 | A=01010 B=01010 Cin=1 | Sum=10101 Cout=0
```

Fig. 38. Outputs on terminal

C. Waveform Analysis

As shown in the simulation waveforms above, the design was verified using the following specific test vectors defined in the testbench:

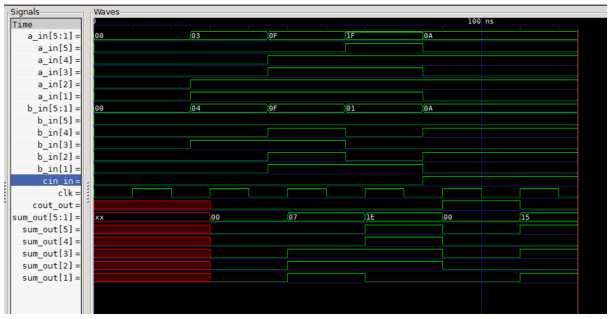

Fig. 39. Gtkwave

- **Test Case 1 (Basic Addition):** Inputs $A = 3$ (00011_2) and $B = 4$ (00100_2) with $C_{in} = 0$.
  Result: $Sum = 7$ (00111_2), $C_{out} = 0$.
- **Test Case 2 (Mid-Range):** Inputs $A = 15$ (01111_2) and $B = 15$ (01111_2) with $C_{in} = 0$.
  Result: $Sum = 30$ (11110_2), $C_{out} = 0$.
- **Test Case 3 (Overflow/Carry-Out):** Inputs set to maximum $A = 31$ (11111_2) and $B = 1$ (00001_2).
  Result: The adder correctly wraps around to $Sum = 0$ (00000_2) and asserts $C_{out} = 1$.
- **Test Case 4 (Carry-In Check):** Inputs $A = 10$ (01010_2) and $B = 10$ (01010_2) with $C_{in} = 1$.
  Result: $Sum = 10 + 10 + 1 = 21$ (10101_2).

The simulation confirms that the structural Verilog model behaves identically to the expected theoretical truth table, validating the logic before physical design.

XV. VIVADO SIMULATIONS

We simulated same test cases as in HDL verilog.

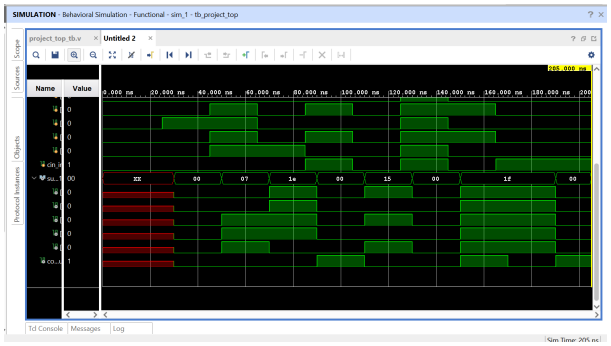

Fig. 40. Wave forms in vivado

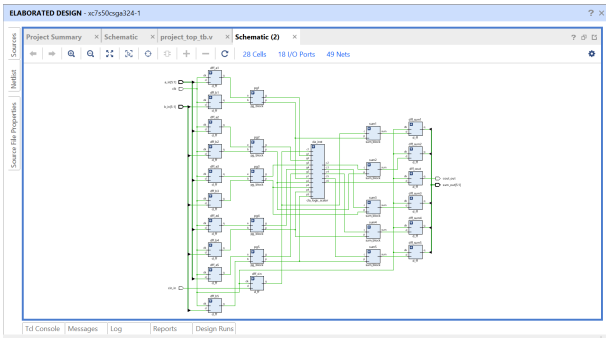

Fig. 41. complete diagram

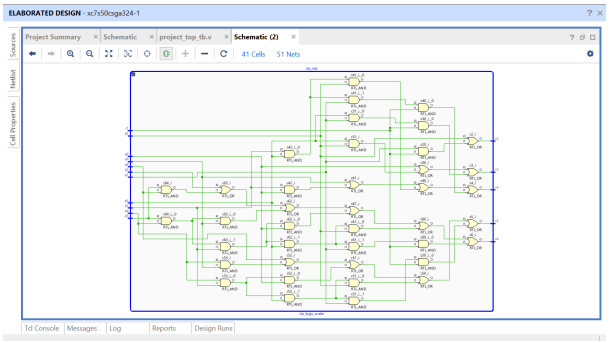

Fig. 42. cla logic

XVI. FPGA Hardware Verification

The proposed 5-bit CLA adder was synthesized and implemented on an FPGA development board to validate its functionality in real hardware. The inputs ($A[5:1]$, $B[5:1]$, C_{in}) were mapped to the on-board dip-switches, and the results (Sum, C_{out}) were observed on the LEDs.

Three specific test cases were performed to verify different aspects of the adder's logic:

- **Test Case 1 (Non-Overlapping Bits):**
  Inputs: $A = 10101_2$ (21), $B = 01010_2$ (10), $C_{in} = 0$.
  Expected Result: $21 + 10 = 31$ (11111_2).
  Observation: The output successfully summed to all ones without generating a carry-out.
- **Test Case 2 (Carry-In Propagation):**
  Inputs: $A = 00010_2$ (2), $B = 00011_2$ (3), $C_{in} = 1$.
  Expected Result: $2 + 3 + 1 = 6$ (00110_2).
  Observation: The adder correctly included the initial carry-in bit in the final summation.
- **Test Case 3 (Overflow/Carry-Out Generation):**
  Inputs: $A = 11110_2$ (30), $B = 11111_2$ (31), $C_{in} = 0$.
  Expected Result: $30 + 31 = 61$.
  Since the 5-bit sum range is 0-31, this results in $C_{out} = 1$ (weight 32) and $Sum = 11101_2$ (29).
  Observation: The Carry-Out LED turned ON, validating the carry look-ahead chain's ability to handle overflow.
- **Test Case (Oscilloscope Verification):**
  Inputs: $A = 11001_2$ (25), $B = 01100_2$ (12), $C_{in} = 0$. We used oscilloscope to find each output connected from fpga.
  Expected Calculation: $25 + 12 = 37$.
  Since 37 exceeds the 5-bit range (max 31), the result is split into the Carry-Out and Sum:
  - $C_{out} = 1$ (Value 32)
  - $Sum = 00101_2$ (Value 5)
  - Total: $32 + 5 = 37$.

  Observation: This specific test vector was used for the oscilloscope capture to analyze the real-time delay and signal integrity of the carry chain during an overflow condition. Below are the simulated pictures.

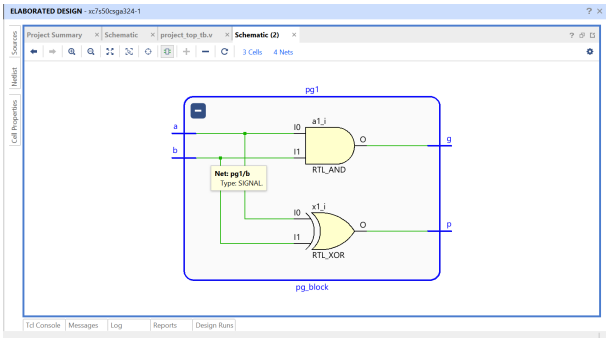

Fig. 43. pg block

XVII. Conclusion

The design of the 5-bit CLA adder was successfully implemented both in ngspice ,magic and verilog , verified through pre-layout and post-layout simulations, and validated on FPGA hardware.

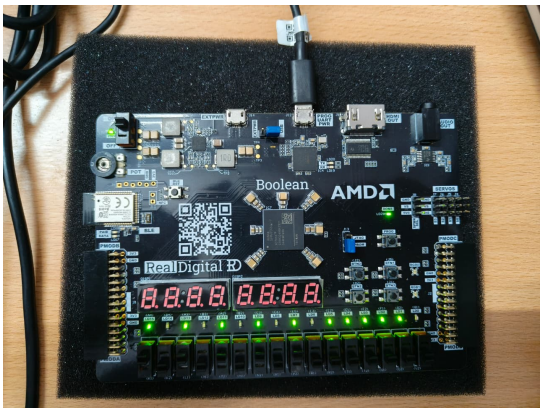

Fig. 44. FPGA Test Case 1: $21 + 10 = 31$

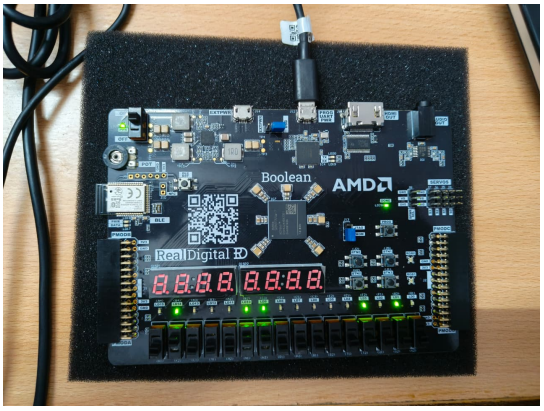

Fig. 45. FPGA Test Case 2: $2 + 3 + 1 = 6$

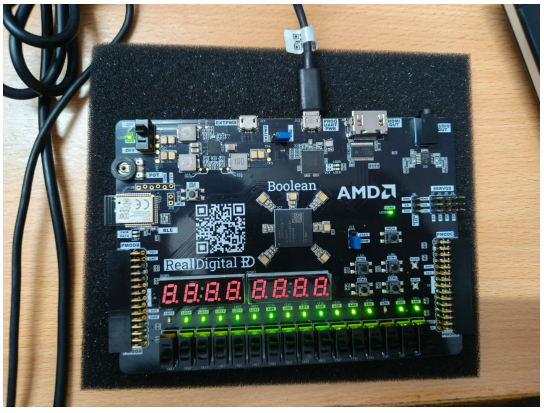

Fig. 46. FPGA Test Case 3: $30 + 31 = 61$ (Cout=1)

Fig. 47. Test case at oscilloscope

Fig. 48. Cout = 1 , S5 = 0

Fig. 49. S4 = 0 , S3 = 1

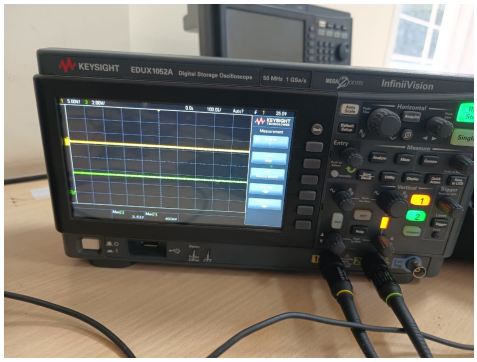

Fig. 50. S2 = 0 , S1 = 1